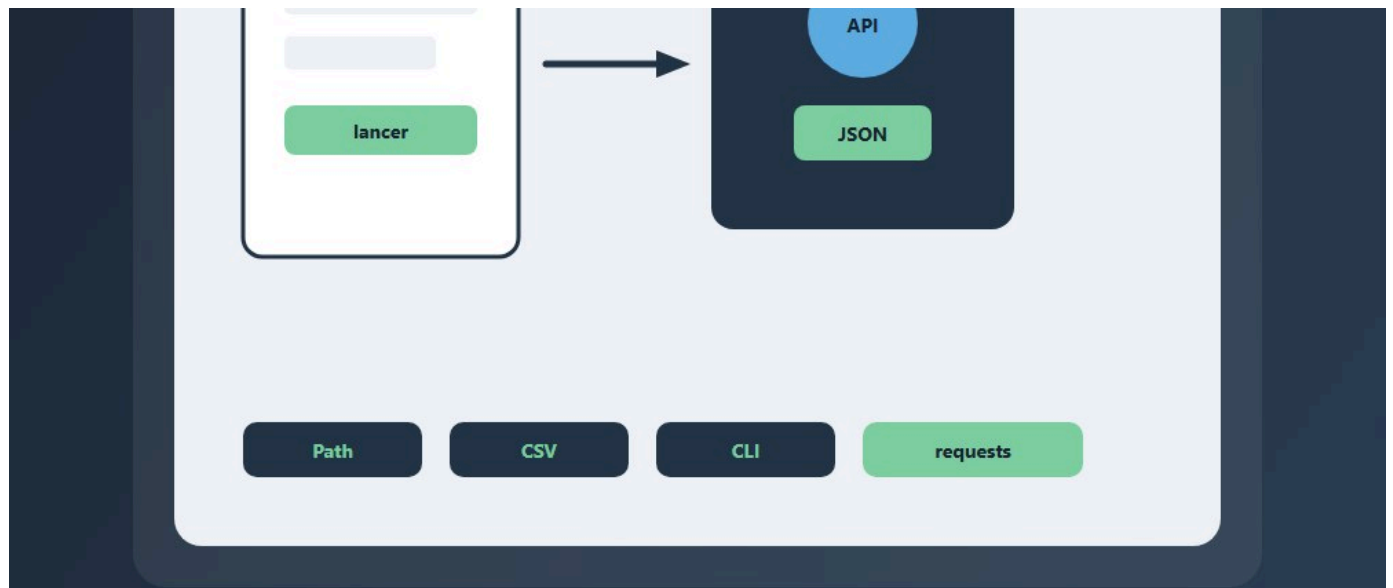




FORMATION INFORMATIQUE - PRATIQUE

Python - Pratique

CSV, CLI, venv, API et bons reflexes : la
suite utile apres les bases Python.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Rappel rapide et carte du livre](#)

- [Ce que tu gardes en tete](#)
- [La carte du livre](#)
- [Un fil rouge](#)
- [Ce dont tu as besoin](#)
- [Comment lire ce livre](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. [Chapitre 2 - pathlib : des chemins sans douleur](#)

- [Creer un Path](#)
- [Existe ? Fichier ? Dossier ?](#)
- [Lire et ecrire du texte](#)
- [Lister un dossier](#)
- [Chemin absolu vs relatif](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. [Chapitre 3 - CSV : lire et ecrire un tableau](#)

- [Un exemple concret : notes](#)
- [Lire avec csv.DictReader](#)
- [Ecrire avec csv.DictWriter](#)
- [Separateur point-virgule](#)
- [Calculer une moyenne](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. [Chapitre 4 - argparse : un script avec des arguments](#)

- [L'idee en une image](#)
- [Premier exemple](#)
- [Types et drapeaux](#)
- [Relier au CSV](#)
- [Arguments positionnels](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. [Chapitre 5 - venv et pip : isoler les paquets](#)

- [Creer un venv](#)
- [L'activer](#)
- [Installer un paquet](#)

- [Verifier](#)
- [.gitignore \(idee\)](#)
- [Mettre a jour pip \(souvent utile\)](#)
- [Plusieurs projets, plusieurs venv](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. [Chapitre 6 - requests GET : appeler une API JSON](#)

- [Installer \(rappel\)](#)
- [Premier appel](#)
- [Meteo : exemple mental](#)
- [Afficher un resume](#)
- [Headers \(idee rapide\)](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

7. [Chapitre 7 - Erreurs HTTP : status, timeout, try/except](#)

- [status code et raise for status](#)
- [Timeout](#)
- [JSON pourri](#)
- [Pattern solide pour un mini outil](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

8. [Chapitre 8 - Variables d'environnement](#)

- [Lire une variable](#)
- [Definir la variable avant de lancer](#)
- [L'idee du fichier .env](#)
- [Charger .env a la main \(mini version\)](#)
- [Config non secrete](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

9. [Chapitre 9 - logging basique](#)

- [Premier logger](#)
- [Niveaux en une idee](#)
- [Logger nomme](#)
- [Ecrire dans un fichier](#)
- [Avec requests / CSV](#)
- [print ou logging ?](#)
- [Exception + log](#)
- [Erreur classique](#)
- [En vrai](#)

- [A toi](#)

10. [Chapitre 10 - datetime pratique](#)

- [Maintenant](#)
- [Formater pour afficher](#)
- [Parser une chaine](#)
- [Ajouter / retirer des jours](#)
- [Filtrer des lignes "du mois"](#)
- [Timestamp de log](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

11. [Chapitre 11 - Regex léger avec re](#)

- [search : y a-t-il un motif ?](#)
- [match vs search vs fullmatch](#)
- [Groupes](#)
- [findall](#)
- [Cas utiles du quotidien](#)
- [Quand ne pas regex](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. [Chapitre 12 - Mini-projet : CLI resume CSV ou API](#)

- [Parcours](#)
- [Squelette argparse](#)
- [Mode notes \(coeur\)](#)
- [Mode meteo \(coeur\)](#)
- [Qualite minimale](#)
- [Organisation minimale du fichier](#)
- [Variante](#)
- [Criteres "c'est fini"](#)
- [A toi](#)

13. [Chapitre 13 - Ce qu'il faut retenir](#)

- [En une minute](#)
- [Habitudes solides](#)
- [Erreurs classiques](#)
- [Suite dans ce livre](#)
- [Mini defi](#)

14. [Chapitre 14 - Atelier : CSV des notes](#)

- [Etapes](#)
- [Criteres de reussite](#)
- [Idee de structure interne](#)
- [Bonus](#)
- [Piege](#)

- [A toi](#)

15. [Chapitre 15 - Atelier : CLI utile](#)

- [Etapas](#)
- [Criteres de reussite](#)
- [Exemple d'appel](#)
- [Bonus](#)
- [Piege](#)
- [A toi](#)

16. [Chapitre 16 - Atelier : appel API meteo](#)

- [Etapas](#)
- [Criteres de reussite](#)
- [Squelette d'extraction](#)
- [Bonus](#)
- [Piege](#)
- [A toi](#)

17. [Chapitre 17 - Tests simples](#)

- [assert : la base](#)
- [Separer le coeur du CLI](#)
- [L'idee pytest](#)
- [Quoi tester en priorite ?](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

18. [Chapitre 18 - Organiser un petit projet](#)

- [Arborescence type](#)
- [requirements.txt](#)
- [README court](#)
- [.gitignore utile](#)
- [Ou mettre le point d'entree ?](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Bonnes pratiques](#)

- [Fonctions courtes, noms clairs](#)
- [Un seul role pour le point d'entree](#)
- [Echecs previsibles](#)
- [UTF-8 et chemins](#)
- [Dependances sous controle](#)
- [Secrets](#)
- [Peu de magie](#)
- [Documenter le "pourquoi" utile](#)
- [Versionne tes donnees d'exemple](#)
- [Petit, mais relancable](#)

- [Eviter la perfection prematuree](#)
- [Petite checklist avant de partager](#)
- [En vrai](#)
- [A toi](#)

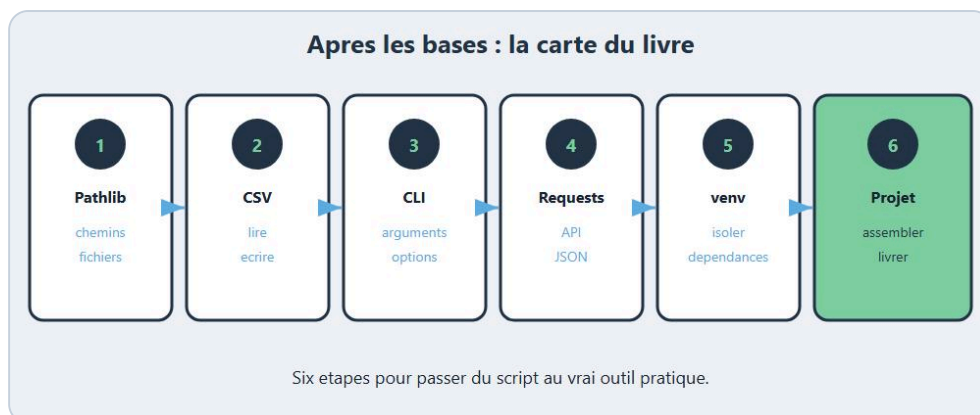
20. [Quiz final](#)

- [Questions](#)
- [Corriges](#)

21. [Bravo](#)

- [Ce que tu sais faire maintenant](#)
- [La suite possible](#)
- [Un conseil pour la route](#)
- [A toi](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : chemins, CSV, CLI, API, projet propre.

Tu as déjà fait les bases. Variables, conditions, boucles, fonctions, listes, dictionnaires, fichiers texte, JSON, modules, exceptions, et même un peu de classes. Si un détail est flou, ce n'est pas grave : on ne va pas tout refaire. Ce livre part du moment où Python devient un outil du quotidien.

Chez DanielCraft, on aime bien une image simple. Les bases, c'était apprendre à conduire sur un parking. La pratique, c'est prendre la route : ouvrir un fichier CSV, lancer un script avec des arguments, appeler une API, isoler ses paquets, et laisser des traces utiles quand ça casse.

Ce que tu gardes en tête

Tu sais écrire une fonction. Tu sais boucler sur une liste. Tu sais ouvrir un fichier texte et lire du JSON. Tu as vu `try/except`. C'est assez pour avancer.

On ne revient pas sur `print`, `if`, `for`, ni sur "c'est quoi un dictionnaire". Si tu bloques sur ces mots, relis le premier livre Python. Ici, on monte d'un cran : des outils standards pour de vrais petits scripts.

La carte du livre

Imagine que tu veux un script qui lit des notes dans un tableau, ou qui demande la météo à une API, ou qui se lance depuis le terminal avec des options. Pour que ça marche vraiment, tu as besoin de plusieurs pièces.

D'abord `pathlib` : des chemins de fichiers sans te battre avec les slash. Ensuite le module `csv` : lire et écrire des tableaux. Puis `argparse` : transformer ton fichier en vrai petit programme en ligne de commande. Un environnement virtuel (`venv`) et `pip` pour installer des paquets sans polluer le reste. `requests` pour appeler une API JSON. Les erreurs HTTP, les variables d'environnement, le `logging`, les dates avec `datetime`, et un peu de regex pour chercher des motifs dans du texte.

Plus tard : un mini-projet qui assemble tout ça, des ateliers concrets, des tests simples, une façon d'organiser un dossier, et des bonnes habitudes. Le quiz vérifie. Le dernier chapitre, c'est pour souffler.

Un fil rouge

On va souvent parler de trois exemples. Un fichier de notes (élève, matière, note) en CSV. Une météo qui arrive en JSON depuis une API. Un script CLI que tu lances dans le terminal avec des arguments. Ces exemples reviennent, pour que tu sentes le progrès.

Tu verras le meme geste sous plusieurs angles : lire des donnees, les resumer, gerer l'echec, rendre le script utilisable par quelqu'un d'autre (toi demain matin compte).

Ce dont tu as besoin

Python 3 installe. Un editeur (VS Code, Cursor, ou autre). Un terminal. Pour la partie API : une connexion internet et un venv. Rien de plus pour suivre. Les paquets hors standard seront installes au fur et a mesure (`requests` , plus tard eventuellement `pytest`).

Comment lire ce livre

Lis dans l'ordre au debut. Chemins, CSV, CLI, venv, requests : ca s'enchaîne. Ensuite erreurs, secrets, logs, dates, regex. Le mini-projet colle les briques. Les ateliers sont la pour faire, pas seulement lire.

Tu n'as pas besoin d'un framework web. Django, Flask, FastAPI viendront plus tard si tu veux. Ici, on reste en Python "outil", clair, dans le terminal et les fichiers.

Erreur classique

Croire que "je connais les bases" = "je sais automatiser mon quotidien". Les bases sont le moteur. La pratique, c'est la boite a outils : CSV, CLI, HTTP, logs. Sans ca, tu reecris tout a la main a chaque fois. Avec ca, tu gagnes du temps chaque semaine.

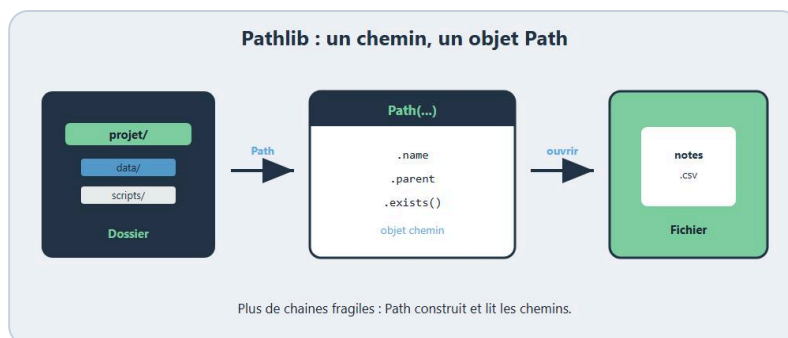
En vrai

Ouvre un vieux script Python que tu as deja écrit. Note ce qui manque pour le rendre utile : lire un tableau ? accepter un argument ? appeler une URL ? isoler les paquets ? Ce livre repond a ca.

A toi

Ecris en trois phrases ce que tu veux construire a la fin. Pas un reseau social. Quelque chose de petit : "resumer mes notes depuis un CSV", "afficher la meteo de ma ville", "un script CLI qui dit bonjour avec mon nom". Garde ce but. On y reviendra.

Chapitre 2 - pathlib : des chemins sans douleur



Path : des chemins clairs, sans galere de slash.

En Python, tu touches souvent a des fichiers. Un CSV de notes, un dossier `data`, un fichier de config. Le piege classique, c'est de coller des chemins a la main avec des chaines et des slash... jusqu'au jour ou ca casse sur Windows, ou sur un autre dossier.

`pathlib` est la solution moderne : tu travailles avec des objets `Path`, pas avec des string brutes. Chez DanielCraft, on resume comme ca : un chemin, c'est un objet, pas un texte magique.

Creer un Path

```
from pathlib import Path

dossier = Path("data")
fichier = Path("data") / "notes.csv"
```

L'operateur `/` assemble les morceaux. Sur Windows comme sur Linux, Python s'occupe du detail. Tu n'as plus a te demander si c'est `\` ou `/`.

Tu peux aussi partir du dossier du script :

```
ici = Path(__file__).resolve().parent
notes = ici / "data" / "notes.csv"
```

`__file__` est le chemin du fichier `.py` en cours. `.parent` remonte d'un cran. C'est utile quand tu lances le script depuis un autre repertoire : tu ne depend plus du "dossier courant" mysterieux.

Existe ? Fichier ? Dossier ?

```
p = Path("data/notes.csv")
print(p.exists())
print(p.is_file())
print(p.parent.is_dir())
```

Avant d'ouvrir, tu peux verifier. Ce n'est pas obligatoire a chaque ligne, mais c'est rassurant au debut. Si le dossier n'existe pas, tu peux le creer :

```
Path("data").mkdir(parents=True, exist_ok=True)
```

`parents=True` cree aussi les dossiers parents manquants. `exist_ok=True` evite une erreur si le dossier est deja la.

Lire et ecrire du texte

Pour un petit fichier texte (ou un JSON en string), `pathlib` a des raccourcis :

```
chemin = Path("data/bonjour.txt")
chemin.write_text("Salut\n", encoding="utf-8")
contenu = chemin.read_text(encoding="utf-8")
print(contenu)
```

Toujours preciser `encoding="utf-8"` quand tu manipules du francais. Sinon, selon la machine, les accents peuvent devenir bizarres.

Pour du binaire (images, etc.), il existe `read_bytes` / `write_bytes`. On reste sur du texte ici.

Lister un dossier

```
dossier = Path("data")
for element in dossier.iterdir():
    print(element.name, element.is_file())
```

Ou filtrer avec un motif :

```
for csv in Path("data").glob("*.csv"):
    print(csv)
```

`rglob` fait la meme chose en recursif (sous-dossiers inclus). Pratique pour trouver tous les `.csv` d'un projet.

Chemin absolu vs relatif

Un chemin relatif depend du dossier ou tu lances Python. Un chemin absolu pointe toujours au meme endroit.

```
p = Path("data/notes.csv")
print(p.resolve())
```

`.resolve()` donne la forme absolue, avec les `..` ranges. Quand tu debogues "fichier introuvable", affiche `resolve()` : souvent tu regardais au mauvais endroit.

Erreur classique

Melanger `os.path.join` partout et `Path` au milieu, sans coherence. Choisis `pathlib` pour les nouveaux scripts. Autre classique : oublier que le dossier courant n'est pas toujours le dossier du script. Utilise `Path(__file__).parent` quand le fichier doit etre "a cote" du code.

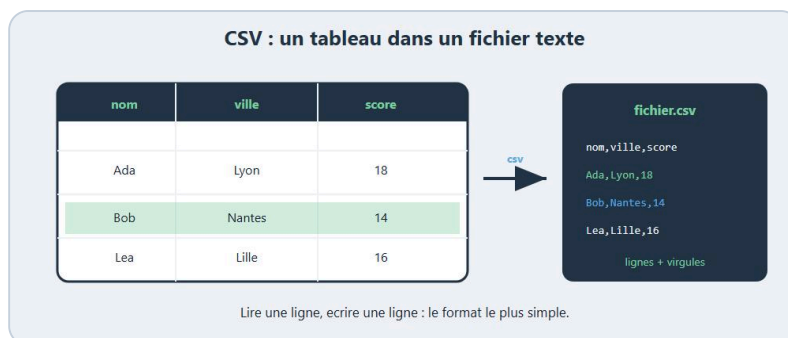
En vrai

Cree un dossier `data` a cote d'un script. Ecris `bonjour.txt` avec `write_text`. Relis-le. Affiche le chemin absolu. Change de repertoire dans le terminal et relance : si tu as bien ancre le chemin sur `__file__`, ca continue de marcher.

A toi

Ecris un petit script qui cree `data/notes.csv` vide (ou avec une ligne d'en-tete), en utilisant uniquement `Path`. Pas de `os.path`. Le geste compte : on branchera le vrai CSV au chapitre suivant.

Chapitre 3 - CSV : lire et ecrire un tableau



CSV : un tableau dans un fichier texte.

Un fichier CSV, c'est un tableau en texte. Chaque ligne est une ligne du tableau. Les colonnes sont separees par une virgule (parfois un point-virgule en France). Excel et les tableurs adorent ce format. Les scripts aussi.

Tu pourrais decouper a la main avec `.split(",")`. Ca marche jusqu'au jour ou une cellule contient une virgule, ou des guillemets. Le module `csv` de la bibliotheque standard gere ca proprement. Chez DanielCraft, on dit : pour un tableau, utilise l'outil tableau.

Un exemple concret : notes

Imagine `data/notes.csv` :

```
eleve,matiere,note
Alice,maths,14
Bob,maths,11
Alice,francais,16
Bob,francais,13
```

Premiere ligne : les noms de colonnes (en-tete). Ensuite les donnees. Simple, lisible, partageable.

Lire avec `csv.DictReader`

```
import csv
from pathlib import Path

chemin = Path("data/notes.csv")

with chemin.open(encoding="utf-8", newline="") as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        print(ligne["eleve"], ligne["matiere"], ligne["note"])
```

Chaque `ligne` est un dictionnaire. Les cles viennent de l'en-tete. C'est confortable : tu penses en colonnes nommees, pas en indices `0`, `1`, `2`.

Le `newline=""` est une recommandation officielle sous Windows pour le module `csv`. Garde-le. L'`encoding="utf-8"` protege les accents.

Attention : les valeurs restent des chaines. "14" n'est pas encore le nombre 14. Convertis quand tu calcules :

```
note = float(ligne["note"])
```

Ecrire avec csv.DictWriter

```
import csv
from pathlib import Path

chemin = Path("data/notes.csv")
lignes = [
    {"eleve": "Alice", "matiere": "maths", "note": "14"},
    {"eleve": "Bob", "matiere": "maths", "note": "11"},
]

with chemin.open("w", encoding="utf-8", newline="") as f:
    champs = ["eleve", "matiere", "note"]
    ecriture = csv.DictWriter(f, fieldnames=champs)
    ecriture.writeheader()
    ecriture.writerows(lignes)
```

`writeheader()` écrit la première ligne. `writerows` écrit le reste. Tu peux aussi faire `writerow` une ligne à la fois dans une boucle.

Separateur point-virgule

En France, Excel exporte parfois avec ;. Dis-le au lecteur :

```
lecteur = csv.DictReader(f, delimiter=";")
```

Et à l'écriture :

```
ecriture = csv.DictWriter(f, fieldnames=champs, delimiter=";")
```

Si tu ouvres un CSV et que "tout est dans une seule colonne", regarde le séparateur.

Calculer une moyenne

Tu lis, tu accumules, tu divises. Rien de magique :

```
total = 0.0
compte = 0
with chemin.open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        if ligne["eleve"] == "Alice":
            total += float(ligne["note"])
            compte += 1
moyenne = total / compte if compte else 0
print("Moyenne Alice :", moyenne)
```

C'est le cœur de beaucoup de petits scripts "métier" : lire un tableau, filtrer, résumer.

Erreur classique

Ouvrir en mode texte sans `newline=""`. Ou oublier que tout est string. Ou ecrire sans `writeheader` puis s'etonner que `DictReader` ne trouve pas les cle. Autre piege : confondre le nom du fichier et son contenu - un mauvais chemin donne un fichier vide ou une erreur, pas un "CSV casse".

En vrai

Cree `notes.csv` a la main (bloc-notes) avec 4-5 lignes. Lis-le avec `DictReader`. Affiche la moyenne d'un eleve. Puis reecris un nouveau fichier `moyennes.csv` avec deux colonnes `eleve,moyenne`.

A toi

Ecris une fonction `moyenne_eleve(chemin, nom)` qui retourne la moyenne, ou `None` si l'eleve est absent. Teste-la avec ton fichier. On reutilisera ce genre de fonction dans le mini-projet et les ateliers.

Chapitre 4 - argparse : un script avec des arguments



Un script que tu lances avec des options.

Tu lances Python avec `python mon_script.py`. C'est bien. Encore mieux : `python mon_script.py --fichier data/notes.csv --eleve Alice`. Le script lit des options. Tu n'as plus à modifier le code pour changer le fichier ou le nom.

`argparse` fait partie de la bibliothèque standard. Il parse la ligne de commande, affiche une aide (`-h`), et refuse les options invalides. Chez DanielCraft, on aime les scripts qui s'expliquent tout seuls.

L'idée en une image

Ton programme déclare : "j'accepte un fichier, et optionnellement un eleve". L'utilisateur passe ces infos. Toi, tu reçois un objet propre avec des attributs. Plus besoin de fouiller `sys.argv` à la main (sauf cas très simples).

Premier exemple

```
import argparse

parser = argparse.ArgumentParser(description="Resume des notes CSV")
parser.add_argument("--fichier", required=True, help="Chemin vers le CSV")
parser.add_argument("--eleve", default=None, help="Filtrer sur un eleve")
args = parser.parse_args()

print("Fichier :", args.fichier)
print("Eleve :", args.eleve)
```

Lance :

```
python resume.py --fichier data/notes.csv --eleve Alice
```

Ou demande l'aide :

```
python resume.py -h
```

Tu verras la description et les options. C'est déjà "pro" pour un petit outil.

Types et drapeaux

Tu peux demander un entier, un flottant, un booléen style drapeau :

```
parser.add_argument("--seuil", type=float, default=10.0)
parser.add_argument("--verbose", action="store_true", help="Plus de details")
```

Avec `--verbose`, `args.verbose` vaut `True`. Sans, `False`. Avec `--seuil 12`, tu reçois `12.0`. Si quelqu'un met du texte a la place d'un nombre, `argparse` proteste clairement.

Relier au CSV

```
import argparse
import csv
from pathlib import Path

parser = argparse.ArgumentParser(description="Moyenne d'un eleve")
parser.add_argument("--fichier", required=True)
parser.add_argument("--eleve", required=True)
args = parser.parse_args()

chemin = Path(args.fichier)
total = 0.0
compte = 0
with chemin.open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        if ligne["eleve"] == args.eleve:
            total += float(ligne["note"])
            compte += 1

if compte == 0:
    print("Aucune note pour", args.eleve)
else:
    print(f"Moyenne de {args.eleve} : {total / compte:.1f}")
```

Tu sens le pattern : parser les arguments, puis faire le travail. Le CLI n'est pas le coeur metier. C'est la poignée de la porte.

Arguments positionnels

Parfois tu veux `python outil.py notes.csv` sans `--fichier` :

```
parser.add_argument("fichier", help="Chemin CSV")
```

Pas de tirets. L'ordre compte. Melange positionnels et options selon ce qui est le plus naturel a taper.

Erreur classique

Tout mettre en dur dans le code (`fichier = "notes.csv"`) et dupliquer le script pour chaque cas. Ou parser `sys.argv[1]` sans aide ni verification. Autre classique : oublier `required=True` puis planter plus loin avec un `None` mysterieux. Mieux vaut une erreur `argparse` des le debut.

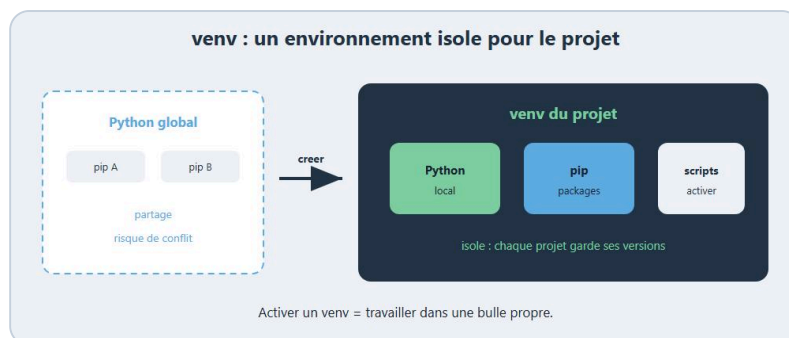
En vrai

Reprends ton lecteur de notes. Ajoute `argparse`. Lance-le trois fois avec des eleves differents, sans toucher au code. Affiche `-h` et lis ce que ca dit. Si l'aide est claire, ton futur toi (ou un collegue) te remerciera.

A toi

Ajoute une option `--seuil` : n'affiche que les notes au-dessus du seuil, ou un message "aucune note au-dessus". Petit, utile, concrete. C'est exactement le genre d'outil qu'on garde.

Chapitre 5 - venv et pip : isoler les paquets



venv : un coin isole pour tes paquets.

Python de base sait beaucoup de choses. Mais pour parler HTTP facilement, on utilisera `requests` (chapitre suivant). `requests` n'est pas dans la bibliotheque standard : tu l'installes avec `pip`.

Si tu installes tout "dans le Python global" de ta machine, les projets se marchent dessus. Une version de bibliotheque pour le projet A casse le projet B. La solution : un environnement virtuel (`venv`) par projet. Chez DanielCraft, regle simple : un projet, un venv.

Creer un venv

Dans le dossier de ton projet :

```
python -m venv .venv
```

Ca cree un dossier `.venv` (le nom est une convention courante). Dedans : un Python isole et un `pip` isole.

L'activer

Sous Windows (PowerShell) :

```
.\.venv\Scripts\Activate.ps1
```

Sous Windows (cmd) :

```
.\.venv\Scripts\activate.bat
```

Sous macOS / Linux :

```
source .venv/bin/activate
```

Quand c'est actif, ton invite de commande montre souvent `(.venv)`. Les commandes `python` et `pip` pointent alors vers le venv.

Pour desactiver : `deactivate`.

Si PowerShell refuse l'execution de scripts, c'est une politique Windows. Tu peux chercher la doc officielle sur l'execution de scripts, ou utiliser `cmd` pour activer. L'idee reste la meme.

Installer un paquet

Avec le venv actif :

```
pip install requests
```

Tu peux figer les versions dans un fichier `requirements.txt` :

```
requests==2.32.3
```

Puis installer d'un coup :

```
pip install -r requirements.txt
```

On approfondira l'organisation du projet plus loin. Pour l'instant, retiens : `pip install` dans le venv, pas "au feeling" dans le Python systeme.

Verifier

```
python -c "import requests; print(requests.__version__)"
```

Si ca affiche une version, `requests` est bien la. Si tu desactives le venv et que tu relances sans le paquet global, ca peut echouer - et c'est normal.

.gitignore (idee)

Le dossier `.venv` ne se partage pas sur Git. Il est lourd et propre a chaque machine. On partage `requirements.txt`, et chacun recree son venv. Si tu utilises Git, ajoute `.venv/` au `.gitignore`.

Mettre a jour pip (souvent utile)

Dans le venv fraichement cree, pip peut etre vieux. Un classique :

```
python -m pip install --upgrade pip
```

Puis installe tes paquets. Moins de messages bizarres, moins de surprises.

Plusieurs projets, plusieurs venv

Projet notes d'un cote, projet meteo de l'autre : deux dossiers, deux `.venv`. Tu actives celui du projet sur lequel tu travailles. Si tu changes de dossier dans le terminal, pense a reactiver (ou a ouvrir un terminal "dans" le bon projet via ton editeur).

Erreur classique

Installer avec un `pip` et lancer avec un autre `python`. Symptome : "j'ai installe requests mais Python dit `ModuleNotFoundError`". Contre-mesure : toujours `python -m pip install ...` avec le meme interpreteur.

```
python -m pip install requests
```

Comme ça, pip et python sont maries.

Autre classique : oublier d'activer le venv et croire que "pip est casse". Ou supprimer le dossier `.venv` à la main puis s'étonner que plus rien ne s'importe : il suffit de recréer le venv et de réinstaller depuis `requirements.txt`.

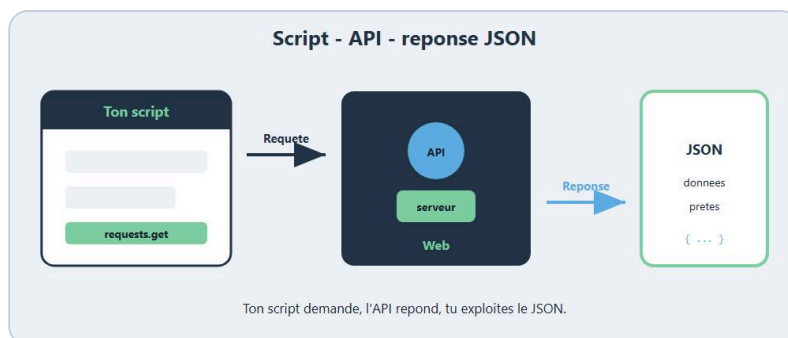
En vrai

Crée un dossier `demo-venv`. Fais `python -m venv .venv`. Active. Installe `requests`. Vérifie l'import. Désactive. C'est un rituel à connaître par cœur.

A toi

Écris un `requirements.txt` minimal avec `requests`. Documente en trois lignes (dans un `README` perso ou un commentaire) : créer le venv, activer, installer. Ton futur toi aime les rituels écrits.

Chapitre 6 - requests GET : appeler une API JSON



Ton script demande des donnees a une API.

Jusqu'ici, tes donnees vivaient dans des fichiers. Maintenant, ton script peut demander des infos ailleurs : une API meteo, un service public, un endpoint de demo.

`requests` est une bibliotheque tres utilisee pour ca. En mode GET, tu dis : "donne-moi cette ressource". Chez DanielCraft, on resume : demander, lire, utiliser.

Installer (rappel)

Dans ton venv :

```
python -m pip install requests
```

Premier appel

```
import requests

url = "https://httpbin.org/json"
reponse = requests.get(url, timeout=10)
print(reponse.status_code)
data = reponse.json()
print(data)
```

Que se passe-t-il ? `requests.get` part chercher l'URL. Tu recois un objet `reponse`. `status_code` est le code HTTP (200 = OK en general). `.json()` parse le corps si c'est du JSON, et te donne un dict ou une liste Python.

Le `timeout=10` evite d'attendre eternellement si le reseau rame. On en reparlera au chapitre erreurs. Prends l'habitude de le mettre des maintenant.

Meteo : exemple mental

Beaucoup d'API meteo renvoient un JSON du genre :

```
{ "ville": "Paris", "temp": 18.2, "ciel": "nuageux" }
```

Ton script fait un GET sur une URL avec la ville en parametre, lit le JSON, puis affiche une phrase. Memes etapes partout. Seules les cles changent.

Avec Open-Meteo (API publique, sans cle pour un usage simple), l'idée ressemble à :

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 48.85,
    "longitude": 2.35,
    "current_weather": "true",
}
reponse = requests.get(url, params=params, timeout=10)
data = reponse.json()
meteo = data["current_weather"]
print("Temperature :", meteo["temperature"])
```

`params` ajoute les paramètres à l'URL proprement (encodage inclus). Tu n'as pas à coller `?latitude=...` à la main.

Afficher un résumé

```
temp = meteo["temperature"]
vent = meteo["windspeed"]
print(f"Il fait {temp} C, vent {vent}")
```

Tu vois le pattern de beaucoup d'outils CLI : appeler, extraire deux-trois champs, afficher clairement. Pas besoin d'une interface graphique pour être utile.

Headers (idée rapide)

Parfois une API demande un en-tête (type de contenu, jeton...). Avec `requests` :

```
headers = {"Accept": "application/json"}
reponse = requests.get(url, headers=headers, timeout=10)
```

On gardera les secrets (jetons) pour le chapitre variables d'environnement. Ne colle jamais une clé API en dur dans un fichier que tu partages.

Erreur classique

Oublier `.json()` et manipuler `reponse.text` en croyant que c'est déjà un dict. Ou appeler sans `timeout`. Ou croire que "pas d'exception" signifie "tout va bien" : un 404 ne lève pas toujours une exception avec `requests` (selon comment tu codes). Le prochain chapitre traite ça.

En vrai

Dans ton `venv`, appelle une URL JSON publique (`httpbin`, `open-meteo`, ou autre dispo chez toi). Affiche le `status_code` et deux clés du JSON. Le but : voir de vraies données arriver dans ton terminal.

À toi

Ecris une fonction `temperature_paris()` qui retourne la température actuelle (float), ou lève une erreur claire si le JSON n'a pas la forme attendue. On solidifiera les erreurs juste après.

Chapitre 7 - Erreurs HTTP : status, timeout, try/except

Appeler le reseau, c'est accepter que ca casse. Pas de wifi. Serveur en panne. URL fausse. Reponse trop lente. JSON inattendu. Un script "pratique" gere ca sans afficher une page de traceback a l'utilisateur.

Chez DanielCraft, regle d'or : echec possible = message clair + details techniques a part (log ou console).

status_code et raise_for_status

```
import requests

reponse = requests.get(url, timeout=10)
print(reponse.status_code)

if reponse.status_code != 200:
    print("Echec HTTP :", reponse.status_code)
else:
    data = reponse.json()
```

Plus court, style requests :

```
reponse = requests.get(url, timeout=10)
reponse.raise_for_status() # leve une erreur si 4xx ou 5xx
data = reponse.json()
```

`raise_for_status()` transforme un 404 ou 500 en exception. Tu peux la capturer dans un `try/except`.

Timeout

```
try:
    reponse = requests.get(url, timeout=5)
    reponse.raise_for_status()
    data = reponse.json()
except requests.Timeout:
    print("Le serveur met trop de temps a repondre.")
except requests.HTTPError as e:
    print("Erreur HTTP :", e.response.status_code)
except requests.RequestException:
    print("Impossible de contacter le serveur.")
```

`RequestException` est la famille large (reseau, HTTP, etc.). Commencer large est OK pour un petit script. Affiner ensuite.

Tu peux aussi ecrire `timeout=(3, 10)` : 3 secondes pour connecter, 10 pour lire. Pour debuter, un seul nombre suffit.

JSON pourri

Parfois le status est 200, mais le corps n'est pas du JSON, ou il manque une cle.

```

try:
    data = reponse.json()
    temp = data["current_weather"]["temperature"]
except ValueError:
    print("La reponse n'est pas du JSON valide.")
except KeyError:
    print("JSON inattendu : cle manquante.")

```

Ne suppose pas que l'API est parfaite. Verifie ce dont tu as besoin.

Pattern solide pour un mini outil

```

def fetch_json(url, params=None):
    try:
        r = requests.get(url, params=params, timeout=10)
        r.raise_for_status()
        return r.json()
    except requests.Timeout:
        raise RuntimeError("Delai depasse") from None
    except requests.RequestException as e:
        raise RuntimeError(f"Echec reseau : {e}") from None

```

Tu centralises. Le reste du programme appelle `fetch_json` et gere un message utilisateur. Moins de copier-coller.

Erreur classique

Avaler toutes les exceptions avec `except:` vide (ou `except Exception: pass`). Tu caches le probleme au lieu de le comprendre. Autre classique : afficher le traceback brut a un utilisateur non technicien. Garde le detail pour toi (`print` debug ou logging au chapitre 9).

En vrai

Appelle une URL volontairement fausse. Puis une URL lente avec un timeout tres court. Observe les exceptions. Branche des messages humains. Casse, puis reparer : c'est comme ca qu'on apprend le reseau.

A toi

Reprends `temperature_paris()` (ou equivalent). Ajoute timeout, `raise_for_status`, et des messages clairs pour timeout / HTTP / JSON. Si les trois chemins d'echec sont propres, ce chapitre est reussi.

Chapitre 8 - Variables d'environnement

Une cle API, un mot de passe, un jeton : ca ne doit pas vivre en clair dans ton code source. Surtout si tu partages le projet, ou si tu le pousse sur Git. La bonne habitude : lire les secrets depuis l'environnement.

`os.environ` donne acces aux variables d'environnement du systeme (ou de ton terminal). Chez DanielCraft, on dit : le code connait le nom de la variable, pas la valeur secrete.

Lire une variable

```
import os

cle = os.environ.get("API_CLE")
if not cle:
    raise RuntimeError("Variable API_CLE manquante")
```

`.get` retourne `None` si absente. Tu peux aussi ecrire `os.environ["API_CLE"]`, qui leve une `KeyError` si elle n'y est pas. Pour un script clair, verifier et afficher un message explicite est souvent mieux.

Definir la variable avant de lancer

Sous Windows PowerShell :

```
$env:API_CLE = "valeur-secrete"
python mon_script.py
```

Sous cmd :

```
set API_CLE=valeur-secrete
python mon_script.py
```

Sous macOS / Linux :

```
export API_CLE=valeur-secrete
python mon_script.py
```

La variable existe pour cette session de terminal. Ce n'est pas magique dans le fichier `.py`.

L'idée du fichier `.env`

Beaucoup de projets utilisent un fichier `.env` a la racine :

```
API_CLE=valeur-secrete
VILLE_DEFAULT=Paris
```

Ce fichier n'est pas du Python. Des outils (comme la bibliotheque `python-dotenv`) peuvent le charger dans `os.environ` au demarrage. L'idée importante, meme sans installer quoi que ce soit pour l'instant :

Tu ne commits pas `.env` (secrets). Tu commits un `.env.example` avec les noms de variables et des fausses valeurs.

```
API_CLE=remplace-moi
VILLE_DEFAULT=Paris
```

Comme ça, un collègue sait quoi configurer sans voir tes vrais secrets.

Charger .env a la main (mini version)

Pour comprendre le principe, sans dependance :

```
from pathlib import Path
import os

def charger_env(chemin=".env"):
    p = Path(chemin)
    if not p.exists():
        return
    for ligne in p.read_text(encoding="utf-8").splitlines():
        ligne = ligne.strip()
        if not ligne or ligne.startswith("#") or "=" not in ligne:
            continue
        nom, valeur = ligne.split("=", 1)
        os.environ.setdefault(nom.strip(), valeur.strip())
```

`setdefault` ne recouvre pas une variable deja definie dans le systeme. Utile.

En pratique, pour un vrai projet, `python-dotenv` fait ca plus completement. L'essentiel est le reflexe : secrets dehors du code.

Config non secrete

Tout n'est pas secret. Une ville par default, un seuil, un mode debug : variables d'environnement ou arguments CLI, selon ce qui est le plus pratique. Les secrets, eux, passent presque toujours par l'environnement (ou un gestionnaire de secrets plus avance, hors sujet ici).

Erreur classique

Ecrire `API_CLE = "sk-xxxxx"` en haut du fichier et pousser sur GitHub. Une fois public, considere la cle brulee : revoque-la. Autre classique : logger la cle "pour debug". Ne jamais imprimer un secret.

En vrai

Cree une variable `MON_NOM` dans ton terminal. Lis-la dans un script et affiche "Bonjour, ...". Puis simule l'absence de variable et affiche une erreur claire.

A toi

Prepares un `.env.example` pour un futur script meteo ou API (noms de variables seulement). Ajoute `.env` a ton `.gitignore` mental (et reel si tu utilises Git). Le muscle "pas de secret dans le code" se construit maintenant.

Chapitre 9 - logging basique

`print` est parfait pour apprendre. Pour un script qui vit un peu, `logging` est plus souple : niveaux (info, warning, error), format, et possibilite d'ecrire dans un fichier sans changer toute la logique.

Chez DanielCraft, on garde `print` pour l'affichage utilisateur du CLI, et `logging` pour ce qui se passe "sous le capot".

Premier logger

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s - %(message)s",
)

logging.info("Demarrage du script")
logging.warning("Fichier presque vide")
logging.error("Echec de l'appel API")
```

Tu verras des lignes avec le niveau. Si tu mets `level=logging.WARNING`, les `info` disparaissent. Pratique : en prod tu baisses le bruit, en debug tu montes le detail.

Niveaux en une idee

`DEBUG` : detail pour developper. `INFO` : deroulement normal. `WARNING` : bizarre mais on continue. `ERROR` : ca a rate. `CRITICAL` : tres grave.

Pour debuter, `INFO` + `ERROR` suffisent souvent. Ajoute `DEBUG` quand tu chasses un bug.

Logger nomme

Dans un projet a plusieurs fichiers, on evite le logger racine partout. Pattern courant :

```
import logging

logger = logging.getLogger(__name__)

def moyenne(valeurs):
    logger.debug("Nb valeurs : %s", len(valeurs))
    if not valeurs:
        logger.error("Liste vide")
        return None
    return sum(valeurs) / len(valeurs)
```

`%s` et les arguments separees : le message n'est formate que si le niveau est actif. Habitude propre.

Configure `basicConfig` une seule fois, cote point d'entree (`main`), pas dans chaque module.

Ecrire dans un fichier

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="app.log",
    encoding="utf-8",
)
```

Utile pour un script lance la nuit, ou pour relire apres coup. Tu peux aussi cumuler console + fichier avec des handlers (un cran plus avance : regarde la doc quand tu en auras besoin).

Avec requests / CSV

```
logger.info("Lecture de %s", chemin)
logger.info("Appel API %s", url)
logger.error("HTTP %s sur %s", status, url)
```

Tu retraces l'histoire sans noyer l'utilisateur. Sur le CLI, toi tu affiches : "Impossible de charger la meteo."
Dans le log : le status, l'URL, peut-etre l'exception.

print ou logging ?

Regle simple pour un CLI : ce que l'humain doit lire comme resultat va dans `print` (la moyenne, la temperature). Ce qui explique le deroulement ou l'echec technique va dans `logging`. Tu peux tout mettre en `print` sur un script de 20 lignes. Des que ca grandit, tu seras content d'avoir separe.

Exception + log

```
try:
    data = fetch_json(url)
except RuntimeError as e:
    logging.exception("Echec fetch")
    print("Impossible de recuperer les donnees.")
```

`logging.exception` (dans un `except`) ajoute la traceback au log. L'utilisateur voit une phrase calme. Toi, tu as le detail dans `app.log` ou dans la console de debug.

Erreur classique

Logger des secrets (jetons, mots de passe). Ou laisser des centaines de `DEBUG` bruyants sans pouvoir les couper. Ou melanger `print` debug partout au point de ne plus voir le vrai resultat utilisateur. Ou appeler `basicConfig` dans trois modules differents et ne plus comprendre quel format gagne.

En vrai

Ajoute trois logs a ton script CSV ou meteo : demarrage, succes, echec. Change le niveau INFO vers WARNING et observe ce qui reste.

A toi

Separe clairement : un `print` pour le resume final, des `logging.info/error` pour le deroulement. Si tu peux couper le bruit en changeant un seul `level=`, c'est gagne.

Chapitre 10 - datetime pratique

Les dates sont partout : horodatage d'un log, "derniere mise a jour", filtrer des lignes apres une certaine date, afficher un jour lisible en francais. Le module `datetime` de la bibliotheque standard couvre l'essentiel.

On reste pratique. Pas de theorie profonde sur les fuseaux horaires (c'est un monde a part). Juste de quoi etre a l'aise au quotidien.

Maintenant

```
from datetime import datetime, date, timedelta

maintenant = datetime.now()
aujourd_hui = date.today()
print(maintenant)
print(aujourd_hui)
```

`date` = jour sans heure. `datetime` = jour + heure. Souvent tu as besoin de l'un ou de l'autre, pas des deux en meme temps.

Formater pour afficher

```
maintenant = datetime.now()
print(maintenant.strftime("%Y-%m-%d %H:%M"))
print(maintenant.strftime("%d/%m/%Y"))
```

`strftime` transforme une date en texte selon un motif. `%Y` annee sur 4 chiffres, `%m` mois, `%d` jour, `%H` heure 24h, `%M` minutes. Tu retiendras les motifs en les reutilisant.

Pour un CSV ou un nom de fichier, le format ISO `2026-07-24` trie bien dans l'ordre alphabetique. Pratique.

Parser une chaine

Tu lis `"24/07/2026"` dans un fichier. Tu veux un vrai objet date :

```
texte = "24/07/2026"
jour = datetime.strptime(texte, "%d/%m/%Y").date()
print(jour)
```

`strptime` = parse. `strftime` = format. Les deux utilisent le meme langage de motifs. Si le texte ne correspond pas, tu attrapes une `ValueError`.

Ajouter / retirer des jours

```
aujourd_hui = date.today()
dans_une_semaine = aujourd_hui + timedelta(days=7)
hier = aujourd_hui - timedelta(days=1)
print(dans_une_semaine, hier)
```

`timedelta` sait aussi `hours=`, `minutes=`. Ideal pour "expire dans 48h" ou "fichier plus vieux que 30 jours".

Filtrer des lignes "du mois"

Imagine un CSV avec une colonne `jour` au format `2026-07-24` :

```
from datetime import date
import csv
from pathlib import Path

debut = date(2026, 7, 1)
fin = date(2026, 7, 31)

with Path("data/events.csv").open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        j = date.fromisoformat(ligne["jour"])
        if debut <= j <= fin:
            print(ligne)
```

`fromisoformat` lit directement `YYYY-MM-DD`. Si tu controles le format de ton CSV, c'est le plus simple.

Timestamp de log

```
import logging
from datetime import datetime

logging.info("Evenement a %s", datetime.now().isoformat(timespec="seconds"))
```

Ou laisse le format `asctime` du logging faire le travail (chapitre precedent). L'idée : une date lisible aide a enquerer.

Erreur classique

Comparer des chaines `"9/7/2026"` et `"10/7/2026"` en ordre alphabetique... ca ment. Parse en `date`, puis compare. Autre classique : oublier que `strftime` et `strptime` ont besoin du meme motif exact.

En vrai

Affiche la date du jour en `JJ/MM/AAAA`. Calcule la date dans 10 jours. Parse une chaine de ton choix. Trois gestes, beaucoup d'usages.

A toi

Ecris `jours_depuis(texte)` qui prend `"2026-01-15"` et retourne le nombre de jours jusqu'a aujourd'hui. Utile pour "ca fait combien de temps ?".

Chapitre 11 - Regex léger avec re

Une expression régulière (regex), c'est un motif pour chercher ou valider du texte. Exemple : "est-ce que ça ressemble à un email ?", "trouve tous les numéros", "extraire un code postal".

Attention : les regex peuvent devenir un sport extrême. Ici, on reste léger. Quelques cas utiles avec le module `re`. Chez DanielCraft, consigne : si un `.startswith` ou un `in` suffit, n'utilise pas de regex.

search : y a-t-il un motif ?

```
import re

texte = "Contacte-moi : ada@exemple.fr merci"
m = re.search(r"[\w.+-]+@[ \w.-]+\.\w+", texte)
if m:
    print("Email trouve :", m.group())
else:
    print("Pas d'email")
```

Le `r"..."` est une raw string : les backslash restent plus lisibles. `search` trouve la première occurrence. `m.group()` donne le texte matché.

Ce motif email est simplifié (pas une validation légale absolue). Pour un exercice ou un filtre grossier, ça va. Pour de la vraie validation métier, les règles sont plus riches.

match vs search vs fullmatch

`search` : quelque part dans la chaîne. `match` : au début. `fullmatch` : la chaîne entière doit correspondre.

```
print(re.fullmatch(r"\d{5}", "75001")) # code postal FR simplifié
print(re.fullmatch(r"\d{5}", "7500")) # None
```

Pour valider une saisie complète, `fullmatch` est souvent le bon réflexe.

Groupes

Tu veux extraire des morceaux :

```
m = re.search(r"note\s*[:=]\s*(\d+(?:\.\d+)?)", "note: 14.5")
if m:
    print(float(m.group(1)))
```

Les parenthèses capturent. `group(1)` est le premier groupe. Utile pour parser des logs ou des lignes un peu libres.

findall

```
texte = "Alice 14, Bob 11, Chloé 16"
notes = re.findall(r"\d+", texte)
print(notes) # ['14', '11', '16']
```

Tu récupères une liste de chaînes. Convertis ensuite en nombres si besoin.

Cas utiles du quotidien

Vérifier qu'un identifiant ne contient que lettres, chiffres, tirets : `r"^[a-zA-Z0-9_-]+$"`. Chercher une URL grossière : `r"https?:\/\/\S+"`. Remplacer des espaces multiples : `re.sub(r"\s+", " ", texte).strip()`.

```
propre = re.sub(r"\s+", " ", " trop d'espaces ").strip()
print(propre)
```

Quand ne pas regex

Compter des lignes CSV : module `csv`. Parser du JSON : `json`. Vérifier une extension de fichier : `Path(p).suffix == ".csv"`. Les regex brillent sur du texte irrégulier. Sur des formats structures, utilise l'outil du format.

Erreur classique

Écrire une regex illisible de trois lignes pour un problème soluble en deux `split`. Ou oublier `r"..."` et se battre avec les `\` escapes. Ou croire qu'une regex email "parfaite" existe en dix caractères.

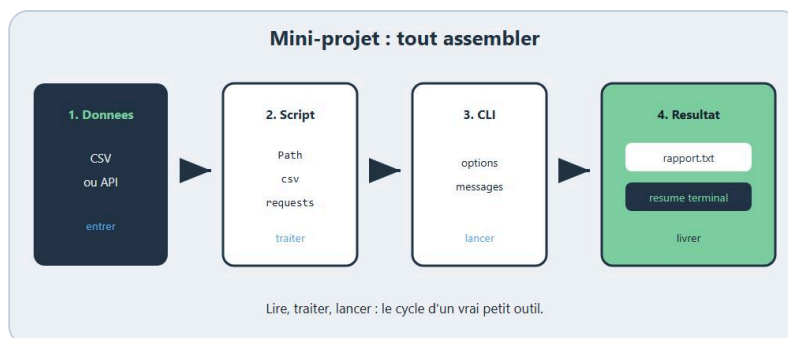
En vrai

Prends un paragraphe avec un email et un numéro à 5 chiffres. Extraie les deux avec `search` / `findall`. Puis valide une saisie code postal avec `fullmatch`.

A toi

Écris `extraire_emails(texte)` qui retourne une liste (peut-être vide). Teste avec 0, 1, 2 emails dans la chaîne. Simple, utile, et tu as touché le cœur de `re` sans te noyer.

Chapitre 12 - Mini-projet : CLI resume CSV ou API



Mini-projet : CLI + donnees + resume clair.

On assemble. Objectif : un petit outil en ligne de commande qui, selon le mode, lit un CSV de notes ou appelle une API meteo, puis affiche un resume clair. Avec arguments, gestion d'erreurs, et un log discret.

Tu n'as pas besoin que ce soit parfait. Il faut que les deux chemins marchent, et que l'echec soit humain.

Parcours

1. 1. Cree un dossier de projet avec un venv et `requests` installe.
2. 2. Place un `data/notes.csv` d'exemple.
3. 3. Ecris `resume.py` avec argparse : `--mode notes|meteo`, et les options utiles.
4. 4. Mode notes : `--fichier` + `--eleve` -> moyenne.
5. 5. Mode meteo : latitude/longitude (ou ville fixe) -> temperature.
6. 6. Erreurs : fichier manquant, eleve inconnu, HTTP rate, timeout.
7. 7. `print` pour l'utilisateur, `logging` pour le detail.

Squelette argparse

```

import argparse
import logging

def build_parser():
    p = argparse.ArgumentParser(description="Resume notes CSV ou meteo API")
    p.add_argument("--mode", choices=["notes", "meteo"], required=True)
    p.add_argument("--fichier", help="CSV pour le mode notes")
    p.add_argument("--eleve", help="Nom de l'eleve")
    p.add_argument("--lat", type=float, default=48.85)
    p.add_argument("--lon", type=float, default=2.35)
    return p
  
```

Tu branches ensuite :

```
def main():
    logging.basicConfig(level=logging.INFO, format="%(levelname)s - %(message)s")
    args = build_parser().parse_args()
    if args.mode == "notes":
        commande_notes(args)
    else:
        commande_meteo(args)

if __name__ == "__main__":
    main()
```

Le `if __name__ == "__main__"` evite d'exécuter `main` si le fichier est importé. Habitude saine.

Mode notes (coeur)

Reutilise `pathlib` + `csv.DictReader`. Calcule la moyenne. Si aucune ligne : message clair. Si le fichier n'existe pas : "Fichier introuvable : ..." et un `logging.error`.

Mode meteo (coeur)

`requests.get` avec timeout, `raise_for_status`, extraction de la température. En cas d'échec : "Impossible de récupérer la météo." + log de l'exception.

Qualité minimale

Pas de traceback brut comme seul résultat. Pas de secret en dur. Aide `-h` lisible. Noms de fonctions clairs (`commande_notes`, `moyenne_eleve`, `fetch_meteo`). Un résumé en une ou deux lignes à l'écran.

Exemple de sorties attendues :

```
Moyenne de Alice : 15.0 (4 notes)
Il fait 18.2 C (lat=48.85, lon=2.35)
```

Organisation minimale du fichier

Même dans un seul `resume.py`, range dans cet ordre : imports, logging config dans `main`, fonctions métier (`moyenne_eleve`, `fetch_meteo`), fonctions CLI (`commande_notes`, `commande_meteo`), puis `main`. Tu te retrouves plus vite. Plus tard, tu pourras couper en plusieurs fichiers (chapitre organisation).

Variante

Ajoute `--seuil` en mode notes pour lister les notes au-dessus du seuil. Ou `--json` pour afficher un mini objet JSON au lieu d'une phrase. Bonus, pas obligatoire.

Critères "c'est fini"

Tu peux expliquer en trente secondes comment lancer les deux modes. Un ami peut lire `-h` et comprendre. Tu as cassé au moins deux choses volontairement (mauvais fichier, mauvais réseau) et les messages tenaient la route. Le code n'a pas de clef en dur.

A toi

Code les deux modes. Casse volontairement le chemin CSV. Casse l'URL ou coupe le reseau. Verifie les messages. Remets tout. Si les chemins heureux et malheureux sont propres, le mini-projet est reussi. Chez DanielCraft, c'est ce genre de petit outil qu'on garde sous le coude.

Chapitre 13 - Ce qu'il faut retenir

Tu as grimpe d'un cran. Voici la carte en version poche.

En une minute

`pathlib` : chemins en objets, `/` pour assembler, `read_text` / `write_text`. `csv` : `DictReader` / `DictWriter` pour les tableaux. `argparse` : options CLI et aide `-h`. `venv` + `pip` : isoler les paquets, `requirements.txt`. `requests` : `GET`, `.json()`, `params`, `timeout`. Erreurs HTTP : `status_code`, `raise_for_status`, `try/except`. Secrets : `os.environ`, idee `.env`, jamais en clair dans le code. `logging` : niveaux et traces. `datetime` : formater, parser, `timedelta`. `re` : `search`, `fullmatch`, `findall` avec `moderation`.

Habitudes solides

1. 1. Un `venv` par projet
2. 2. Toujours un `timeout` sur le reseau
3. 3. Messages humains a l'ecran, details en log
4. 4. Secrets hors du code source
5. 5. CSV avec `newline=""` et `encoding="utf-8"`
6. 6. Chemins ancrees avec `Path(__file__).parent` quand besoin
7. 7. Un role clair par fonction

Erreurs classiques

Installer un paquet dans le mauvais Python. Oublier que les cellules CSV sont des strings. Croire qu'un GET sans exception signifie succes. Coller une cle API dans le fichier. Ecrire une regex la ou un `Path.suffix` suffisait. Tout mettre dans un seul `main` de 200 lignes.

Suite dans ce livre

Ateliers concrets (CSV, CLI, API), puis tests simples, organisation de projet, bonnes pratiques, quiz, bravo. Tu n'as pas besoin d'etre parfait. Tu as besoin d'etre capable de lire un tableau, lancer un script avec des options, et appeler une API sans paniquer.

Mini defi

Sans regarder tes notes, ecris de memoire : ouvrir un CSV avec `DictReader`, et un `requests.get` avec `timeout` + `raise_for_status`. Compare ensuite. Les trous montrent ce qu'il faut relire.

Chapitre 14 - Atelier : CSV des notes

Objectif : un petit pipeline local. Tu crées un CSV, tu calcules des moyennes par élève, tu écris un second CSV resume. Pas d'API ici. Que des fichiers et du Python clair.

Étapes

1. Crée le dossier `atelier-csv` avec un sous-dossier `data`.
2. Écris `data/notes.csv` avec l'en-tête `eleve,matiere,note` et au moins 6 lignes (2 élèves, 3 matières).
3. Écris `resume_notes.py` qui utilise `pathlib` et `csv`.
4. Lis toutes les notes. Accumule somme et compte par élève (dictionnaire de dictionnaires, ou deux dicos).
5. Affiche dans le terminal chaque élève et sa moyenne arrondie à 1 décimale.
6. Écris `data/moyennes.csv` avec les colonnes `eleve,moyenne,nb_notes`.
7. Gère le cas fichier manquant avec un message clair (pas seulement le traceback).

Critères de réussite

- Relancer le script deux fois de suite recrée `moyennes.csv` sans planter.
- Les moyennes sont correctes (vérifie à la main sur un élève).
- Le code utilise `DictReader` / `DictWriter`, pas un `.split(",")` maison.
- Encoding UTF-8 partout.

Idee de structure interne

```
def lire_notes(chemin):
    ...

def moyennes_par_eleve(lignes):
    ...

def ecrire_moyennes(chemin, moyennes):
    ...
```

Tu testes mentalement chaque morceau. Au chapitre tests, tu pourras poser des `assert` sur `moyennes_par_eleve` sans toucher aux fichiers.

Bonus

Ajoute `argparse` : `--fichier` pour le CSV source, `--sortie` pour le CSV moyennes. Valeurs par défaut vers `data/notes.csv` et `data/moyennes.csv`.

Piege

Les notes en string. Convertis avec `float` (ou `int` si tu es sûr des entiers). Si une ligne a une note invalide, décide : ignorer avec un warning, ou arrêter avec une erreur claire. Les deux sont défensables ; l'important est d'avoir choisi. Autre piège : oublier `writeheader` et te retrouver avec un CSV sans colonnes nommées.

A toi

Quand `moyennes.csv` est correct, ouvre-le dans un tableur si tu veux. Le pont Python <-> tableur, c'est souvent ça le vrai gain.

Chapitre 15 - Atelier : CLI utile

Objectif : transformer un script "en dur" en outil avec arguments. Tu dois pouvoir changer de fichier et d'eleve sans editer le code.

Etapas

1. Reprends (ou recree) un script qui calcule la moyenne d'un eleve depuis un CSV.
2. Ajoute `argparse` avec `--fichier` (obligatoire) et `--eleve` (obligatoire).
3. Ajoute `--seuil` optionnel (float). Si present, affiche aussi combien de notes sont $\geq$ seuil pour cet eleve.
4. Ajoute `--verbose` (`store_true`). En verbose, logue chaque ligne retenue (via `logging` ou `print` clairement marque).
5. Verifie que `python script.py -h` affiche une aide comprehensible en francais.
6. Teste au moins trois lancements : eleve connu, eleve inconnu, fichier inexistant.

Criteres de reussite

- L'aide `-h` explique chaque option sans jargon inutile.
- Eleve inconnu -> message du type "Aucune note pour X", code retour non zero possible (`sys.exit(1)`).
- Fichier manquant -> message clair.
- Aucune valeur secrete, aucun chemin absolu grave dans le code (chemins passes en arguments).

Exemple d'appel

```
python moyenne_cli.py --fichier data/notes.csv --eleve Alice --seuil 12 --verbose
```

Bonus

Accepte un argument positionnel `fichier` a la place de `--fichier`, et garde `--eleve`. Habitue-toi aux deux styles.

Piege

Tout valider "plus loin" dans le code alors qu'`argparse` peut deja exiger les options. Si `--eleve` manque, mieux vaut le refus immediat d'`argparse` qu'un `None` mysterieux au milieu du calcul.

A toi

Montre a quelqu'un (ou a toi-meme demain) uniquement l'aide `-h` et un exemple d'appel. Si on comprend sans lire le source, l'atelier est reussi. C'est l'esprit des petits outils DanielCraft : utilisables, pas seulement "techniquement corrects".

Chapitre 16 - Atelier : appel API meteo

Objectif : un script qui recupere la meteo actuelle via une API JSON publique, affiche un resume, et gere proprement les echecs reseau. Travaille dans un venv avec `requests`.

Etapes

1. Cree un dossier `atelier-api` et un venv. Installe `requests`. Ecris un `requirements.txt`.
2. Choisis une API simple (Open-Meteo convient : pas de cle pour un usage de base).
3. Ecris `meteo.py` avec argparse : `--lat`, `--lon` (defaults Paris ok), eventuellement `--timeout`.
4. Fais un GET avec `params` et `timeout`.
5. Verifie le succes (`raise_for_status` ou test du status).
6. Extrais temperature (et vent si dispo). Affiche une phrase claire.
7. Attrape timeout et erreurs reseau : message utilisateur + log.
8. Ajoute une variable d'environnement optionnelle `METEO_TIMEOUT` pour surcharger le timeout si presente.

Criteres de reussite

- `python meteo.py -h` est lisible.
- Un appel normal affiche temperature (et unite).
- Un timeout tres court (ex: 0.001) produit un message propre, pas seulement un mur de traceback.
- Aucune cle en dur dans le fichier (meme si l'API n'en demande pas : prends le reflexe).

Squelette d'extraction

```
data = reponse.json()
meteo = data["current_weather"]
temp = meteo["temperature"]
print(f"Temperature : {temp} C")
```

Adapte les clees si tu changes d'API. L'important est le parcours, pas la marque du service.

Bonus

Mode `--brut` qui affiche le JSON indenté (`json.dumps(data, indent=2)`) pour debug. Ou ecriture d'une ligne dans `data/historique.csv` avec date + temperature (`datetime` + `csv`).

Piege

Oublier que la structure JSON peut changer. Protege les `KeyError`. Ne suppose pas que "200" implique les clees que tu aimes.

A toi

Lance le script pour deux paires lat/lon differentes. Compare. Si tu sens que c'est devenu un outil (et plus un brouillon), l'atelier est bon.

Chapitre 17 - Tests simples

Quand une fonction calcule une moyenne ou parse une date, tu veux pouvoir verifier sans relancer tout le CLI a la main. Les tests, c'est ca : des mini-verifications automatiques.

On reste simple. D'abord `assert`. Ensuite l'idée de `pytest`. Pas besoin d'une usine.

assert : la base

```
def moyenne(valeurs):
    if not valeurs:
        return None
    return sum(valeurs) / len(valeurs)

assert moyenne([10, 20]) == 15
assert moyenne([]) is None
print("Tests OK")
```

Si un `assert` echoue, Python leve `AssertionError` et s'arrete. Si tout passe, tu vois "Tests OK". Pour de petites fonctions pures (entree -> sortie, sans reseau), c'est deja enorme.

Separer le coeur du CLI

Le piege : tout coller dans `main` avec `argparse`. Difficile a tester. Mieux :

```
def moyenne_eleve(lignes, nom):
    notes = [float(l["note"]) for l in lignes if l["eleve"] == nom]
    if not notes:
        return None
    return sum(notes) / len(notes)
```

Tu testes `moyenne_eleve` avec une liste de dicts en memoire. Pas besoin de fichier. Le CLI, lui, lit le CSV puis appelle la fonction.

L'idée pytest

`pytest` est un outil populaire. Tu installes dans le venv : `pip install pytest`. Tu ecris un fichier `test_moyenne.py` :

```
from resume import moyenne_eleve

def test_moyenne_simple():
    lignes = [
        {"eleve": "Alice", "note": "10"},
        {"eleve": "Alice", "note": "20"},
    ]
    assert moyenne_eleve(lignes, "Alice") == 15

def test_eleve_absent():
    assert moyenne_eleve([], "Bob") is None
```

Puis tu lances :

pytest trouve les fonctions `test_...`, les execute, et resume les succes/echec. Plus confortable que des assert disperses quand le projet grandit.

Tu n'es pas obligé d'installer pytest aujourd'hui. Comprends surtout : fonctions testables + verifications automatiques.

Quoi tester en priorite ?

Les calculs. Les parsers (date, validation regex legere). Les branches "vide / absent". Moins urgent au debut : tout le reseau (ca demande des mocks). Pour l'API, tu peux tester une fonction qui extrait la temperature d'un dict JSON fictif.

```
def temperature_depuis(data):  
    return float(data["current_weather"]["temperature"])  
  
assert temperature_depuis({"current_weather": {"temperature": 18.2}}) == 18.2
```

Erreur classique

Tester uniquement le chemin heureux. Ajoute le cas vide, le cas absent, une note non numerique si tu la geres. Autre classique : asserts sur le texte exact d'un message utilisateur qui change souvent - prefere tester la valeur de retour.

En vrai

Extrais une fonction pure de ton atelier CSV. Ecris 3 asserts. Fais-en echouer un volontairement pour voir le message, puis reparer.

A toi

Si tu es a l'aise, installe pytest et deplace tes asserts dans `test_*.py`. Sinon, garde un fichier `checks.py` lanceable. L'habitude compte plus que l'outil.

Chapitre 18 - Organiser un petit projet

Un script unique, c'est bien pour apprendre. Des qu'il y a CSV + API + CLI + tests, un peu d'ordre evite le chaos. Pas besoin d'architecture d'entreprise. Juste un dossier clair.

Chez DanielCraft, on vise : un nouvel arrivant comprend en 30 secondes ou cliquer.

Arborescence type

```
mon-outil/  
  .env/  
  .gitignore  
  .env.example  
  requirements.txt  
  README.md  
  data/  
    notes.csv  
  src/  
    mon_outil/  
      __init__.py  
      cli.py  
      notes.py  
      meteo.py  
  tests/  
    test_notes.py
```

Variante plus simple, tout a plat au debut :

```
mon-outil/  
  .env/  
  requirements.txt  
  resume.py  
  notes_lib.py  
  data/  
  tests/
```

L'important : separer donnees, code, tests. Ne pas melanger le CSV de prod et les brouillons sans nom.

requirements.txt

```
requests==2.32.3  
pytest==8.3.0
```

Les versions exactes (pin) rendent les installs reproductibles. Pour un apprentissage, pinner au moins les paquets critiques.

Installation :

```
python -m venv .env  
# activer le venv  
python -m pip install -r requirements.txt
```

README court

Quatre blocs suffisent : a quoi sert l'outil, comment installer, comment lancer (2 exemples), ou mettre sa cle si besoin (renvoi a `.env.example`). Ecris pour un humain presse, pas pour un roman.

`.gitignore` utile

```
.venv/  
.env  
__pycache__/  
*.pyc  
.pytest_cache/  
app.log
```

Tu ignores le virtuel, les secrets, le cache Python, les logs locaux.

Ou mettre le point d'entree ?

`cli.py` ou `resume.py` parse les arguments et appelle des fonctions. `notes.py` / `meteo.py` portent la logique. Les tests importent la logique, pas le parseur complet (sauf tests d'integration plus tard).

Erreur classique

Un dossier `Nouveau dossier (3)` avec `script_final_v2_REEL.py`. Ou installer sans `requirements.txt` et ne plus savoir quoi recreer sur une autre machine. Ou committer `.venv` (lourd, inutile, parfois fragile).

En vrai

Reprends ton mini-projet. Range-le dans une arborescence simple. Ajoute `requirements.txt`, `.env.example`, un README de 15 lignes. Supprime les fichiers morts.

A toi

Ecris la structure choisie sur papier (ou dans le README) avant de deplacer les fichiers. Dix minutes d'ordre economisent une heure de "ou j'avais mis ca ?".

Chapitre 19 - Bonnes pratiques

Ce chapitre ne rajoute presque pas de syntaxe. Il solidifie des reflexes. Tu peux le relire de temps en temps, comme une checklist amicale.

Fonctions courtes, noms clairs

`moyenne_eleve` dit ce qu'elle fait. `calc2` non. Une fonction qui lit un CSV, appelle le reseau, formate trois messages et ecrit un fichier... c'est trop. Decoupe. Chez DanielCraft, on prefere trois fonctions ennuyeuses a une fonction heros.

Un seul role pour le point d'entree

`main` parse, appelle, affiche. La logique metier vit ailleurs. Benefice : tests plus simples, CLI plus mince.

Echecs previsibles

Fichier absent, JSON inattendu, reseau down : tu les as deja vus. Traite-les tot. Messages en francais simple. Details techniques dans les logs. Code retour non zero (`sys.exit(1)`) quand le script est utilise dans une chaine d'outils.

UTF-8 et chemins

Toujours `encoding="utf-8"` pour le texte francais. Preferer `pathlib`. Eviter les chemins absolus graves dans le code (`C:\Users\...`) : passe-les en argument ou relative au projet.

Dependances sous controle

Venv. `requirements.txt`. `python -m pip`. Pas de "j'ai installe globalement, tant pis". Moins de mysteres entre machines.

Secrets

Variables d'environnement. `.env` ignore par Git. `.env.example` partageable. Jamais de jeton dans un screenshot de cours ni dans un log.

Peu de magie

Les regex, les one-liners cryptiques, les imports circulaires : souvent impressifs, rarement gentils pour le futur toi. Clairete > densite.

Documenter le "pourquoi" utile

Un README court. Une docstring sur une fonction non evidente. Pas besoin de commenter `i += 1`. Commente l'intention quand le code seul ne suffit pas : "Open-Meteo renvoie la temp en C sous `current_weather`".

Versionne tes donnees d'exemple

Un petit `data/notes.exemple.csv` aide. Evite de casser le CSV "reel" a chaque test. Copie, travaille, compare.

Petit, mais relancable

Un bon script se relance demain sans que tu te souviennes de tous les details. L'aide `-h`, le README, et des exemples d'appel dans un commentaire en haut du fichier aident enormement. Ecris pour ton futur toi fatigue.

Eviter la perfection prematuree

Pas besoin de microservices pour un CSV de notes. Pas besoin de dix fichiers pour trente lignes. Organise assez pour rester lisible. Si tu passes plus de temps a "architecturer" qu'a faire marcher le cas nominal, simplifie.

Petite checklist avant de partager

Est-ce que `-h` est clair ? Est-ce qu'un fichier manquant est explique ? Est-ce que le venv est documente ? Est-ce qu'un secret traine ? Est-ce qu'un test minimal existe pour le coeur du calcul ? Si oui, ton script merite d'etre montre.

En vrai

Relis ton mini-projet avec cette checklist. Corrige deux choses seulement, mais pour de vrai. Les bonnes pratiques arrivent par couches, pas par sermon.

A toi

Choisis une habitude (timeout, venv, ou secrets) et applique-la systematiquement a tous tes scripts cette semaine. Une habitude ancree vaut mieux que dix intentions.

Quiz final

Pas de piège. Relis une fois si besoin. L'idée : vérifier que tu peux avancer sans paniquer.

Questions

1. 1. Pourquoi préférer `pathlib.Path` à une string de chemin colle à la main ?
 - A) Parce que Path colorie le terminal
 - B) Pour manipuler des chemins proprement (joindre, vérifier, lire) sans galérer avec les slash
 - C) Parce que CSV l'exige
1. 2. Avec `csv.DictReader`, chaque ligne est surtout :
 - A) Un entier
 - B) Un dictionnaire dont les clés viennent de l'en-tête
 - C) Une image
1. 3. À quoi sert `argparse` ?
 - A) À installer des paquets
 - B) À définir et lire les arguments d'un script en ligne de commande
 - C) À créer des pages web
1. 4. Un environnement virtuel (`venv`), c'est surtout pour :
 - A) Isoler les paquets d'un projet
 - B) Remplacer Git
 - C) Compiler le système
1. 5. Quelle commande installe souvent les dépendances listées ?
 - A) `python -m pip install -r requirements.txt`
 - B) `csv install requirements`
 - C) `pathlib --install`
1. 6. Dans `requests`, que fait souvent `reponse.json()` ?
 - A) Ouvre Excel
 - B) Parse le corps JSON en structures Python (dict/list)
 - C) Crée un venv
1. 7. Pourquoi mettre un `timeout` sur un GET ?
 - A) Pour colorier les logs
 - B) Pour éviter d'attendre indéfiniment si le réseau/serveur rame
 - C) Parce que CSV l'impose
1. 8. `raise_for_status()` sert surtout à :
 - A) Ignorer les erreurs

- B) Lever une exception si le code HTTP indique une erreur (4xx/5xx)

- C) Formater une date

1. 9. Ou stocker une cle API plutot que dans le code source ?

- A) Dans une variable d'environnement (idee .env hors Git)
- B) Dans le nom du fichier CSV
- C) Dans un commentaire `print`

1. 10. Le niveau `logging.INFO` par rapport a `DEBUG` :

- A) Affiche en general moins de detail que DEBUG
- B) Remplace `argparse`
- C) Chiffre les secrets

1. 11. `strftime` sert a :

- A) Installer pytest
- B) Formater une date/heure en chaine selon un motif
- C) Lire un CSV

1. 12. Une bonne raison d'utiliser `re.search` :

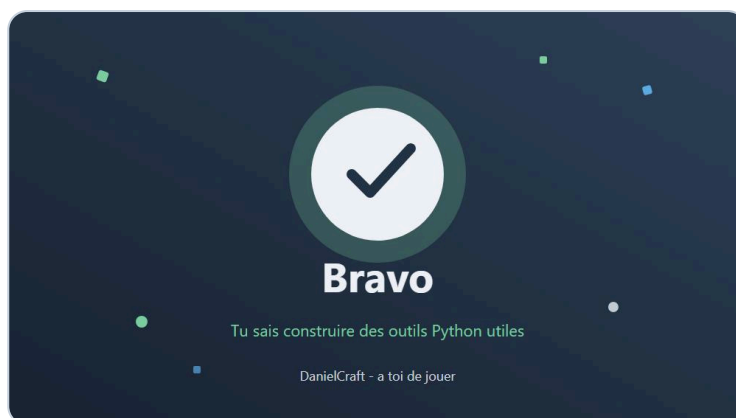
- A) Remplacer `pathlib` pour lister des dossiers
- B) Chercher un motif dans un texte irregulier (ex: email approximatif)
- C) Activer le `venv`

Corriges

1-B, 2-B, 3-B, 4-A, 5-A, 6-B, 7-B, 8-B, 9-A, 10-A, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es pret pour de vrais petits outils. Sinon, relis les chapitres lies (chemins, CSV, CLI, reseau, secrets), sans dramatiser.

Bravo



Bravo. Tu sais faire des scripts Python utiles.

Tu as fini Python - Pratique. Chez DanielCraft, on marque le moment : tu ne fais plus seulement tourner des exercices de bases. Tu sais lire un tableau CSV, lancer un script avec des arguments, isoler tes paquets, appeler une API JSON, gerer un echec reseau, et ranger un petit projet sans panique.

Ce que tu sais faire maintenant

Manipuler des chemins avec `pathlib`. Lire et ecrire du CSV proprement. Construire un CLI avec `argparse`. Creer un `venv` et installer avec `pip`. Faire un GET avec `requests`, avec timeout et verification HTTP. Garder les secrets hors du code. Logger sans noyer l'utilisateur. Formater des dates. Utiliser une regex avec moderation. Poser un test simple sur une fonction pure.

La suite possible

Un vrai petit outil perso cette semaine : suivi de notes, export CSV, rappel meteo du matin, nettoyage de fichiers. Plus tard, si tu veux : API web (Flask/FastAPI), automatisation plus poussee, bases de donnees, packaging. Mais tu as le socle pratique qui porte tout ca : fichiers, CLI, HTTP, hygiene de projet.

Un conseil pour la route

Garde un depot "outils" avec trois scripts qui te servent vraiment. Ameliore-les quand ca frotte. Le muscle se construit en servant ton quotidien, pas en collectionnant des tutoriels.

A toi

Ecris en trois lignes : ton prochain script, la donnee qu'il lit (fichier ou API), et le premier argument CLI que tu vas ajouter. Puis ouvre l'editeur.

Bravo. Vraiment.

Tu as les briques. Maintenant, construis.